Temperature	Mean Deviation	Solve Rate (%)	Avg. Token Length
0.0	6.0%	**46.6%**	1778
0.3	6.3%	**46.6%**	**1782**
0.6	6.2%	46.5%	1778
1.0	**5.6%**	46.4%	1765

Table 4: Effect of sampling temperature on length control and accuracy for L1-Exact. Despite changing the temperature, the model shows similar errors in length control and similar Pass@1 solve rates throughout all temperature settings.

remains stable (5.6-6.3%) across all temperature settings, with marginal differences in token length and solve rate. These results indicate that L1's length control capabilities are robust to sampling parameters, allowing flexible deployment across different inference settings without compromising performance or adherence to length constraints. Further, given that we average across four target lengths $n_{gold} \in \{512, 1024, 2048, 3600\}$, the observed average token lengths (1765-1782) closely match the ideal average of 1796 tokens.

A.5 Prior Reasoning Models do not follow length constraints in Prompts

Requested Length	512	1024	2048	3600
Generated Length	5491	6072	5846	6421

Table 5: Average length generated by DeepScaleR-24K for different requested lengths. The model does not follow the user-specified length constraint.

A natural question is whether existing reasoning models can follow length constraints through simple prompting without specialized training. To investigate this, we explicitly prompted DeepScaleR-24K to generate responses of specific token lengths (512, 1024, 2048, and 3600) using clear prompts like "Think for exactly N tokens." As shown in Table 5, the model consistently generates outputs close to 6000 tokens regardless of the requested length, demonstrating a complete insensitivity to length instructions.

A.6 Supervised Fine-Tuning does not learn length control

We further compare how our proposed RL recipe compares to SFT. For training models with supervised finetune, we first generate responses without any length control, then measure their token lengths and re-prompt with the measured token counts to create instruction-response pairs with explicit length requirements. For training the models, we perform a hyperparameter search over Learning Rates: 1e-6, 1e-5, 1e-4 and Batch Sizes: 256, 512. We use validation loss for early stopping.

After training with this SFT data, we evaluated adherence by prompting for specific lengths. The results are presented in Table 6. The results highlights that the model still failed to follow length constraints, producing very long outputs regardless of the requested target.

Requested Tokens	512	1024	2048	4096
Actual Tokens	21388	22749	21426	20903

Table 6: SFT-only model does not learn to follow token-length constraints despite being trained on relabeled data with explicit length requirements.

These results, along with Table 5, indicate that offline methods such as SFT are ineffective at inducing precise length control in reasoning models. We hypothesize two reasons: (1) the narrow distribution of output lengths for a given question leads the model to ignore length

instructions, and (2) without an online length-sensitive reward, the model does not acquire the capability to adjust its reasoning to meet target budgets.

A.7 Ablating Reward Functions in LCPO

We evaluate how different objective functions affect the performance and token adherence of LCPO. Specifically, we ablate Equation 2 using the following variants:

1. **L1-Max Single Objective**: During the training of L1-Max, we originally employed both objectives from Eq. 2 and Eq. 1. In this variant, we exclusively use the former.
2. **L1-Max Addition**: This variant implements a simple addition of correctness and length penalty objectives instead of multiplication. The objective function is: $r(y, y_{gold}, n_{gold}) = \mathbb{I}(y = y_{gold}) - \alpha \cdot \left| n_{gold} - n_y \right|$.
3. **L1-Max Sigmoid**: This variant applies a sigmoid function to penalize length violations using the objective: $r(y, y_{gold}, n_{gold}) = \mathbb{I}(y = y_{gold}) \cdot \sigma(\alpha \cdot (n_{gold} - n_y))$, where $\sigma(x) = \frac{1}{1+e^{-x}}$ is the sigmoid function.

We train each variant for 120 RL steps, initializing from the same L1-Exact checkpoint. Figure 17 presents the budget violation rates across these variants. While all variants exhibit similar violation rates across different token budgets, L1-Max Addition achieves a 0% budget violation rate consistently. This perfect adherence, however, occurs because the model collapses to generating extremely short CoTs, which trivially satisfies the maximum length constraint which is undesirable behavior.

Figure 15 demonstrates this effect by showing performance versus token length for all variants. LCPO-Max Addition performs substantially worse than the other three variants. For clarity, we exclude this variant in Figure 16. The remaining three variants perform similarly without any statistically significant differences. We ultimately select the standard L1-Max for all our experiments due to its simplicity and the potential advantages of its dual objective approach.

A.8 Parallel vs Sequential Scaling

Figure 18 compares how sequential scaling (increasing CoT length) and parallel scaling (Majority Voting) vary in performance for the same incurred cost (total tokens generated). There are a few interesting observations from the graph: First, parallel scaling is almost always worse than sequential scaling, when compared at the same token budget. Second, parallel scaling can provide significant improvements, to all reasoning models (eg, more than 10% in some cases). Third, while Qwen-2.5-1.5B-Math, barely benefits from majority voting, L1-Max (1024 tokens), despite using similar number of generation tokens in each task, benefits significantly. This is noteworthy, since for many cases, it is desirable to have low token lengths, because of both reduced latency, and avoiding the quadratic cost of generation. In such cases, LCPO-Max clearly severs as a better alternative to parallel scaling of non-reasoning models. Finally, LCPO-Max (3600 tokens) matches the performance of DeepScaleR-4K baseline across parallel scaling. We do note a small dip in the last point, but this is because only 1 seed was used for parallel scaling, and as such the difference is not statistically significant.

A.9 LCPO scales to larger reasoning models

We further scaled the LCPO recipe to DeepSeek-R1-Distill-7B model. Results are presented in Figure 19. We observe the same trends as with the 1.5B model: high controllability, low budget-violation rates, and strong performance relative to length-controlled baselines across budgets. These results highlight the effectiveness and potential of LCPO to scale to even larger reasoning models.

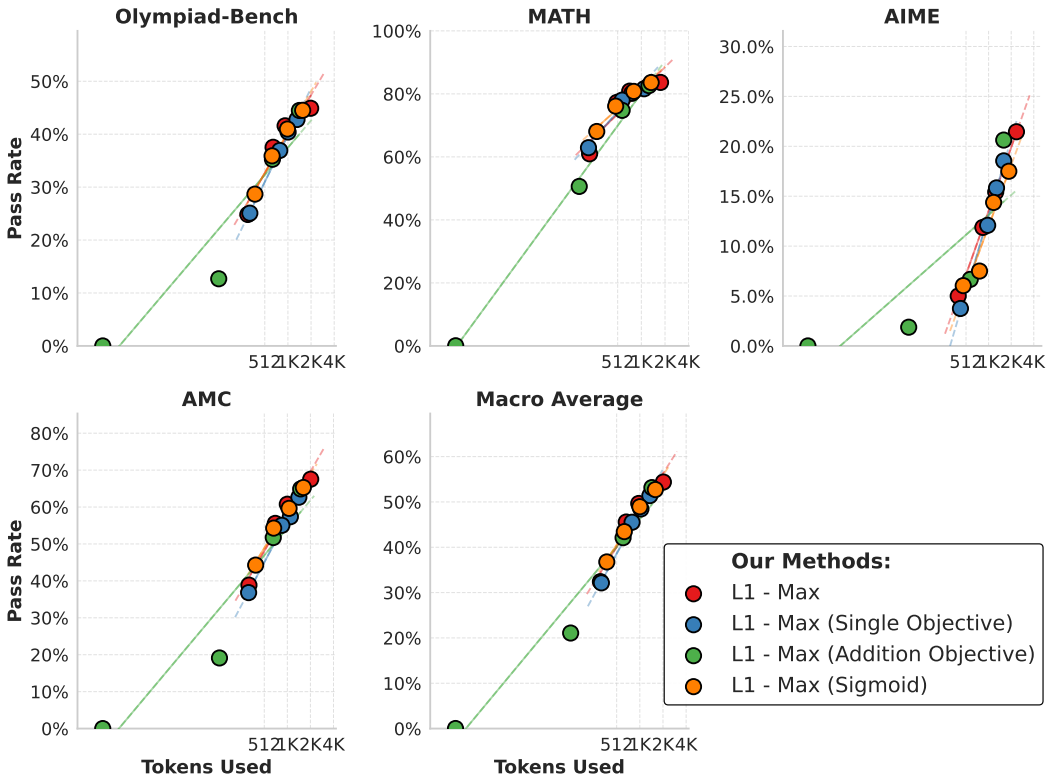

Figure 15: Performance of different Reward Functions (RFs) used to train L1-Max.

A.10 Qualitative Examples

In Figures 20 and 21, we show two examples of L1-Exact solving the same problem but with different requested generation lengths: 512 and 3600 tokens. While the model with 512 tokens is unable to solve the problem correctly, the model with longer generation arrives at the correct answer. We refer interested readers to our website, which has more such interesting examples.